**České vysoké učení technické v Praze**
Fakulta elektrotechnická

# Bludiště na rotačním stolku

Technická zpráva

**Autoři:** **[DOPLNIT: jména všech tří členů týmu]**
**Předmět:** **[DOPLNIT: kód a název předmětu]**
**Zvolená varianta:** C – trajektorie vyžadující pohyb mimo rovinu, bonus za oba směry otáčení
**Pracoviště:** Bosch SR450 (4-DoF SCARA), kamera Basler, rotační stolek
**Datum:** 9. listopadu 2026

**Abstrakt**

Zpráva popisuje řešení, ve kterém průmyslový robot Bosch SR450 provlékne obruč bludištěm umístěným na otáčejícím se stolku. Poloha a otáčení desky s bludištěm se určí z kamery pomocí dvou ArUco značek. Robot pak předem spočítá časový plán, ve kterém je střed obruče v každém kroku na tyči, a plán projede. Plán hlídá také kolizi robota s bludištěm. Pokud je bludiště mimo dosah robota nebo kamera nevidí obě ArUco značky, program to zjistí a nahlásí ještě před zahájením pohybu. Řešení odpovídá variantě C zadání (trajektorie mimo rovinu, rotační stolek) a bylo otestováno pro oba směry otáčení stolku.

# 1 Úvod

Úkolem semestrální práce je provléct obruč, kterou drží robot na svém konci, bludištěm (tyčí) umístěným na rotačním stolku. Stolek se během pokusu otáčí nejpomalejší rychlostí; směr otáčení (vlevo/vpravo) volí cvičící a zvládnutí obou směrů je oceněno bonusem. Bludiště je pro snadnou lokalizaci opatřeno ArUco značkami a jeho rozměry i rozměry obruče jsou k dispozici v repozitáři zadání (github.com/pilcsimo/ROB1-2025-hw).

## 1.1 Pravidla a kritéria úspěchu

Podle zadání platí pro každý pokus:

- **úspěšný pokus:** robot dostane obruč přes bludiště (tyčku) a na konci trajektorie je obruč níže než 4 cm nad podložkou s ArUco značkami;
- **časový limit:** nejvýše 2 minuty na jeden pokus;
- **neúspěšné pokusy:** nejvýše 3 (např. dotyk stěny bludiště nebo tyčky);
- **diskvalifikace:** kolize robota se stolem, stolkem nebo se sebou samým; dotyk s bludištěm po splnění úlohy je přípustný;
- **nedosažitelná poloha:** je-li bludiště umístěno tak, že ho robot nemůže bezpečně dosáhnout, musí to program zjistit a nahlásit (např. výpisem) ještě před zahájením pohybu. Takový pokus se nepočítá jako neúspěšný.

## 1.2 Zvolená varianta

Zadání nabízí varianty lišící se tvarem bludiště (tabulka 1). S 4-DoF robotem Bosch lze splnit varianty A1–C (nejvýše 15 bodů + bonus), protože robot nemůže naklánět obruč – má jen 4 stupně volnosti.

| varianta | popis | max. bodů |
|---|---|---|
| A1 | rovná tyčka bez rotačního stolku | 2 |
| A2 | rovná tyčka položená kdekoliv na rotačním stolku; nedosažitelnou polohu nahlásit | 8 |
| B | složitější trajektorie v rovině (na rotačním stolku) | 12 |
| **C** | **trajektorie vyžadující pohyb mimo rovinu (na rotačním stolku)** | **15** |
| D | trajektorie s natočením obruče (pouze 6-DoF) | 18 |
| **bonus** | **řešení funguje pro oba směry otáčení stolku (varianty A2–D)** | **+2** |

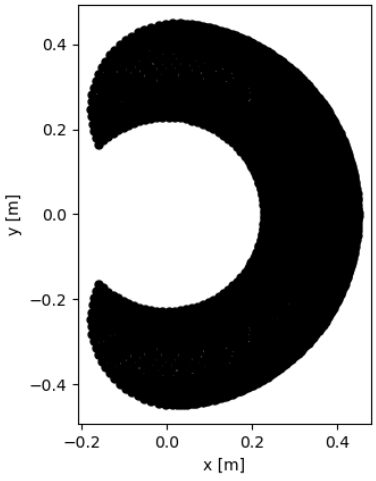
Tabulka 1: Varianty zadání; zvolená varianta je vyznačena tučně.

Cvarianta.png
(doplnit obrazek)

Obrázek 1: Bludiště varianty C.

### 1.3 Pracoviště

- **robot Bosch SR450**: SCARA se 4 stupni volnosti. Klouby $q_1$ a $q_2$ jsou rotační (ramena 250 mm a 200 mm), $q_3$ je posuvný (výška), $q_4$ otáčí koncovým bodem kolem svislé osy. Ovládá se knihovnou `ctu_bosch_sr450` (dokumentace stanice). Limity kloubů: $q_1 \in \pm 85°$, $q_2 \in \pm 120°$, $q_3 \in \langle -0{,}33;\ 0{,}005 \rangle$ m, $q_4 \in \langle -20°;\ 1080° \rangle$. Dosažitelná oblast koncového bodu je na obrázku 2;
- **kamera Basler** umístěná nad pracovní plochou, přibližně kolmo k ní;
- **rotační stolek** s podložkou, na které je bludiště (tyč) a dvě ArUco značky (ID 1 a 2, slovník $4 \times 4$) v protilehlých rozích;
- **obruč** na držáku pod přírubou robota; střed obruče je 135 mm od osy hřídele.

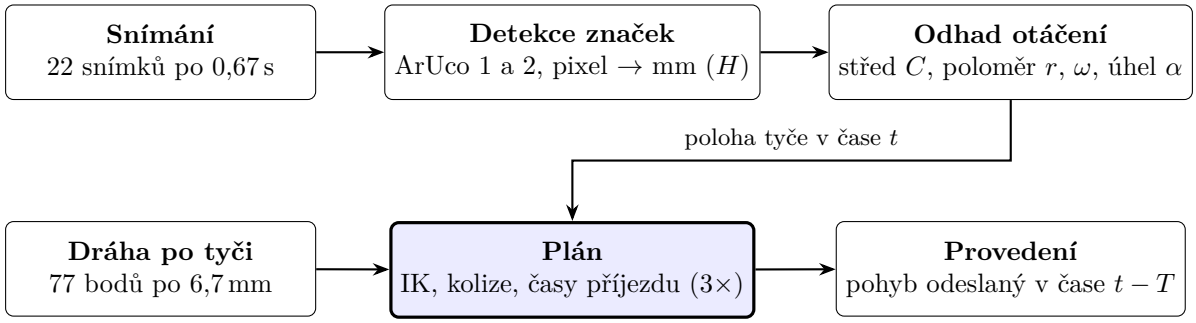



Obrázek 2: Dosažitelné polohy koncového bodu robota v rovině $xy$ základny (v rámci limitů kloubů $q_1$ a $q_2$).

## 2 Přehled řešení

Řešení má dvě fáze. Nejdřív robot z kamery zjistí, jak se deska s bludištěm otáčí, a předem spočítá celý plán: pro každý krok čas příjezdu a konfiguraci, při které je střed obruče na tyči.

Potom plán projede, bez dalšího pozorování kamerou (obrázek 3).




Obrázek 3: Průběh řešení: robot nejdřív spočítá celý plán, pak ho projede.

Odhad otáčení dává plánu polohu každého bodu tyče v libovolném čase; plán k ní hledá konfiguraci robota a čas příjezdu. Homografie $H$ pochází z kalibrace (kapitola 3) a doby pohybu z modelu změřeného předem (kapitola 7.1).

# 3 Kalibrace kamery a obruče

Kalibrace převádí pixely kamery na souřadnice $XY$ v základně robota [mm]. Využívá samotnou obruč: robot ji postaví do 12 známých poloh, kamera najde její střed v obraze a ze dvojic „pixel $\leftrightarrow$ poloha z kinematiky“ se spočítá homografie.

## 3.1 Poloha obruče vůči koncovému bodu

Knihovna robota vrací polohu koncového bodu (osy hřídele). Střed obruče je od ní posunutý o vektor $D$ v souřadnicích koncového bodu:

$$c = e + R(\varphi)\, D, \qquad D = 0{,}135\,\mathrm{m} \cdot (\cos 189^\circ,\ \sin 189^\circ), \tag{1}$$

kde $e$ je poloha koncového bodu z přímé kinematiky a $\varphi = q_4$ je natočení koncového bodu. U tohoto robota se $\varphi$ měří absolutně od osy $x$ základny, ne relativně k ramenům. Délka 135 mm je vzatá z dokumentace, úhel 189° vyšel z fitu kalibračních dat (kapitola 7.2). Při plánování se používá opačný vztah $e = c - R(\varphi)D$: pro zvolené natočení obruče se dopočítá, kde musí být koncový bod.

## 3.2 Oprava nuly kloubu $q_2$

Fit kalibračních dat ukázal, že úhel, který robot hlásí pro kloub $q_2$ (loket), se od skutečného liší o stálou hodnotu:

$$q_2^{\mathrm{skut}} = q_2^{\mathrm{povel}} + \Delta q_2, \qquad \Delta q_2 = 0{,}06\,\mathrm{rad} \approx 3{,}4^\circ. \tag{2}$$

Bez opravy se střed obruče posune až o asi 1 cm, a to různě podle natočení ramen. Inverzní kinematika knihovny počítá se skutečným úhlem, takže robotu se posílá $q_2 - \Delta q_2$; při kontrole kolize se naopak poloha ramene počítá ze skutečného úhlu $q_2 + \Delta q_2$ (konstanta `Q2_OFF`).

## 3.3 Snímání kalibračních poloh

Skript `calibrate.py` projede 12 konfigurací (`CALIB_Q`) ve výšce $q_3 = -0{,}23$. V každé počká 0,5 s na ustálení, uloží snímek a skutečné kloubové souřadnice. Polohy jsou rozprostřené po celém záběru kamery a vybrané tak, aby rameno robota nezakrývalo obruč.

### 3.4 Výpočet homografie

Na každém snímku se střed obruče najde Houghovou transformací pro kružnice (`cv2.HoughCircles` po mediánovém filtru). K němu se přiřadí poloha středu obruče z přímé kinematiky:

$$x = 250\cos q_1 + 200\cos(q_1 + q_2) + 135\cos(q_4 + \delta), \tag{3}$$
$$y = 250\sin q_1 + 200\sin(q_1 + q_2) + 135\sin(q_4 + \delta), \tag{4}$$

kde $\delta$ je úhel vektoru $D$. Z 12 dvojic se metodou nejmenších čtverců (`cv2.findHomography`) spočítá homografie $H$, která mapuje pixel na $XY$ v mm. Homografie platí pro rovinu, ve které byla obruč při kalibraci; ta je v rovině značek na desce.

**[DOPLNIT: finální program načítá `H_angle_candidate_mm.npy`; uveďte, z jakých dat a s jakým $\delta$ a $\Delta q_2$ vznikla. Pro konzistenci by měla být spočítaná se stejným modelem obruče jako zbytek programu, např. `H_fit_mm.npy` z `fit_calibration.py`.]**

## 4 Odhad otáčení stolku

Z 22 snímků pořízených v rozestupu 0,67 s (asi 14 s) se určí střed otáčení, poloměr, úhlová rychlost a natočení desky. Z nich lze pro libovolný čas $t$ spočítat, kde bude každý bod tyče. Implementace je ve třídě `image_proc` (`image_processing.py`).

### 4.1 Snímání a čas snímku

Funkce `grab_image()` při každém volání spustí snímání kamery, vezme první snímek, přenese ho (asi 5 Mpx) a snímání zastaví. Snímek tedy vzniká na začátku volání a zbytek je přenos. Jako čas snímku se proto bere začátek volání + `CAMERA_DELAY` (0,03 s), ne střed volání.

### 4.2 Detekce značek

Na každém snímku se detektorem ArUco (`DICT_4X4_50`) najdou značky 1 a 2. Jejich rohy se homografií převedou do roviny stolku a střed značky je průměr rohů. Snímky, na kterých nejsou vidět obě značky (zakryje je rameno), se vynechají. Program pokračuje jen při aspoň 8 platných snímcích. Jinak robot přejede do polohy pro focení a fotí znovu; pokud ani pak nevidí obě značky na 8 snímcích, ukončí se s hlášením „Aruco značky nejsou vidět“ ještě před zahájením jízdy.

### 4.3 Střed otáčení, poloměr a natočení desky

Střed desky $s$ je průměr středů obou značek. Ten při otáčení opisuje kružnici, která se proloží algebraicky metodou nejmenších čtverců:

$$x^2 + y^2 = 2ax + 2by + c \quad \Rightarrow \quad C = (a, b), \quad r = \sqrt{c + a^2 + b^2}. \tag{5}$$

Značky jsou v protilehlých rozích čtverce, takže osa $x$ desky svírá s diagonálou (od značky 2 ke značce 1) úhel 45°. Úhel $\alpha$ mezi osou desky a spojnicí střed otáčení–střed desky je pro tuhou desku stálý. Odhaduje se jako kruhový průměr přes všechny snímky.

### 4.4 Úhlová rychlost a předpověď

Úhel $\theta$ středu desky kolem středu otáčení se rozbalí (`np.unwrap`) a proloží přímkou v čase. Směrnice je úhlová rychlost $\omega$ a absolutní člen je úhel v referenčním čase $t_{\text{ref}}$ (čas posledního platného snímku). Referenční úhel se bere z proložené přímky přes všechny snímky, ne z jediného posledního snímku. Šum jednoho snímku by jinak posunul celou předpověď.

Bod tyče $p_b$, zadaný v souřadnicích desky [mm], je v čase $t$ ve světě v bodě

$$\theta(t) = \theta_{\text{ref}} + \omega\,(t - t_{\text{ref}}), \qquad p_w(t) = C + r\begin{pmatrix}\cos\theta(t)\\ \sin\theta(t)\end{pmatrix} + R\big(\theta(t)+\alpha\big)\,p_b. \tag{6}$$

Výška bodu se nemění: $z = \texttt{Z_PODLOZKA} + z_b$, kde $\texttt{Z_PODLOZKA} = 0{,}27\,\text{m}$ je výška desky v souřadnicích robota.

### 4.5 Geometrie tyče

Tyč je zadaná lomenou čarou v souřadnicích desky [mm], od horního konce dolů a zpět nahoru:

$$(50, 50, 200) \to (50, 50, 150) \to (50, 0, 100) \to (0, 0, 50) \to (0, 0, 20) \to \text{zpět stejnou cestou.}$$

Šikmé úseky leží ve dvou různých svislých rovinách, takže obruč se pohybuje ve všech třech osách – odtud varianta C.

## 5 Plánování a provedení trajektorie

Robot si nejdřív spočítá celý plán: pro každý krok čas příjezdu a kloubovou konfiguraci, při které je střed obruče na tyči. Pak plán projede. Plán s předem spočítanými časy je jednodušší než přepočítávání za jízdy a dobře se ladí.

### 5.1 Rozdělení dráhy

K lomené čáře tyče se na začátek přidá bod 30 mm nad horním koncem (nájezd). Každý úsek se rozdělí na kroky dlouhé nejvýš 6,7 mm (`STEP_MM`). Pro současnou tyč vznikne 77 bodů; body 0–40 vedou dolů (bod 40 je nejnižší, 20 mm nad podložkou), od bodu 41 se obruč vrací nahoru.

### 5.2 Model doby pohybu

Plán potřebuje vědět, jak dlouho bude robot do dalšího bodu jet. Doba pohybu se odhaduje modelem změřeným skriptem `measure.py`: každý kloub se nezávisle posune o několik různých vzdáleností a doba se proloží přímkou. Doba současného pohybu více kloubů je doba nejpomalejšího z nich:

$$T(\Delta q) = \max_j \left( a_j + \frac{|\Delta q_j|}{v_j} \right). \tag{7}$$

Parametry pro relativní rychlost $\texttt{SPEED} = 0{,}5$ jsou v kapitole 7.1.

### 5.3 Inverzní kinematika a výběr řešení

Obruč je osově souměrná, takže na jejím natočení nezáleží a pro jeden bod tyče existuje mnoho konfigurací. Pro každé natočení $\varphi$ z mřížky po 5° se spočítá poloha koncového bodu $e = c - R(\varphi)D$ a zavolá se inverzní kinematika knihovny (`robot.ik`). Ta vrátí až dvě polohy lokte a více variant $q_4$ ($\varphi + 2k\pi$ v rozsahu kloubu). Každé řešení se opraví o $\Delta q_2$ (kapitola 3.2) a vyřadí se řešení mimo limity kloubů a řešení, která kolidují s tyčí (kapitola 6).

Ze zbylých se vybere to nejbližší předchozímu kroku (nejmenší $\|q - q_{\text{prev}}\|$). Po dráze platí dvě omezení:

- $q_4$ **se za krok otočí nejvýš o** 30° (`MAX_DPHI`). Jinak by výběr mohl zvolit stejnou polohu s $q_4$ o 360° jinak a robot by se otočil dokola;
- **loket nesmí přeskočit na druhou stranu**. Přeskok by znamenal velký pohyb celého ramene nad bludištěm.

Pro první bod (nájezd nad tyč) se vybere natočení, při kterém je kolidující díl co nejdál od tyče.

### 5.4 Časování plánu

Kam má robot jet, závisí na čase příjezdu, a čas příjezdu závisí na tom, jak daleko robot jede. Plán se proto počítá třikrát:

1. odhadnou se časy příjezdu $t_i$ do všech bodů;
2. pro každý bod se spočítá poloha tyče v čase $t_i + \texttt{LEAD_T}$ a vybere konfigurace;
3. z modelu doby pohybu se určí rozestupy kroků $1{,}1\,T + 0{,}05\,\mathrm{s}$ a nové časy. Začátek se posune tak, aby robot stihl výpočet a příjezd na první bod.

`LEAD_T` vyrovnává stálé zpoždění mezi skutečnou polohou desky a předpovědí (zpoždění kamery, reakce robota). Kladná hodnota míří tam, kde tyč bude později, záporná dříve. Ve finální verzi se nastavuje podle směru otáčení a polohy stolku (tabulka 2); hodnoty byly nalezeny experimentálně (kapitola 7.3).

| | střed otáčení $y \le 0$ | střed otáčení $y > 0$ |
|---|---|---|
| $\omega < 0$ | $+0{,}05\,\mathrm{s}$ | $0{,}00\,\mathrm{s}$ |
| $\omega \ge 0$ | $-0{,}03\,\mathrm{s}$ | $-0{,}08\,\mathrm{s}$ |

Tabulka 2: Hodnota `LEAD_T` podle směru otáčení stolku a strany, na které stolek leží.

### 5.5 Nedosažitelná poloha

Pokud pro některý bod dráhy neexistuje konfigurace v limitech kloubů bez kolize, výpočet plánu skončí výpisem „Plán selhal: bod $i$ nemá řešení IK …“. Protože některá selhání závisí na natočení stolku v okamžiku příjezdu (kolize dílu u hřídele s tyčí), program pak desku znovu nafotí a plán spočítá ještě jednou. Pokud selže i druhý plán, vypíše „Bludiště je v této poloze nedosažitelné, nepohybuji se.“ a robot se k bludišti vůbec nerozjede. Nedosažitelnost je tak nahlášena ještě před zahájením pohybu, jak požaduje zadání.

### 5.6 Provedení plánu

Každý pohyb se odešle v čase $t_i - T_i$ (`move_to_q`), aby robot dojel přesně v plánovaný čas, a pak se počká na zastavení (`wait_for_motion_stop`). Na konci se vypíše, o kolik se robot od plánu zpozdil.

Nejnižší bod dráhy je 20 mm nad podložkou, tedy pod hranicí 4 cm z kritéria úspěchu. Dráha pak pokračuje po tyči zpět nahoru, takže robot na konci bezpečně vyjede svisle nahoru a vrátí se domů, aniž by obruč táhl přes tyč.

Celý pokus trvá zhruba **[DOPLNIT: změřit; odhad: 15 s focení + asi 1 s plán + asi 40 s jízda]**, tedy pod limitem 2 minut.

## 6 Kolize robota s bludištěm

Obruč sama do tyče nenarazí, pokud je její střed na tyči. Do tyče ale narážel malý svislý váleček na příchytce vedle hřídele. Plán proto vyřazuje konfigurace, ve kterých by se tento díl dostal k tyči.

### 6.1 Model kolidujícího dílu

Ramena a loket jsou podle výkresu robota výš než vršek tyče, takže do ní narazit nemohou. Kolidující díl je přichycený k hřídeli a jezdí nahoru a dolů spolu s obručí. V půdorysu ho modelujeme jako svislý válec (tabulka 3).

| parametr | hodnota | význam |
|---|---|---|
| `HOUSING_OFFSET` | 85 mm | vzdálenost středu dílu od osy hřídele |
| `HOUSING_ANGLE` | 157° | směr od osy hřídele, měřený od směru ramene 2 (loket → hřídel); díl je mezi loktem a hřídelí, 23° vlevo |
| `HOUSING_R` | 35 mm | poloměr dílu + poloměr tyče + rezerva |
| `HOUSING_DZ` | 30 mm | o kolik je spodek dílu výš než rovina obruče |

Tabulka 3: Model kolidujícího dílu.

Díl je pevně na rameni 2, takže se neotáčí s $q_4$. Jeho střed je v bodě $e + \texttt{HOUSING_OFFSET} \cdot (\cos\beta, \sin\beta)$, kde $\beta = q_1 + q_2^{\text{skut}} + \texttt{HOUSING_ANGLE}$.

### 6.2 Kontrola kolize

Tyč se navzorkuje body po 5 mm, včetně svislého kusu od spodního bodu dráhy až na desku, kde je tyč uchycená. Pro kandidátní konfiguraci a čas příjezdu se tyč posune do světa (kapitola 4.4). Kontrolují se jen body tyče, které jsou výš než spodek dílu. Konfigurace koliduje, pokud je některý z nich vodorovně blíž než `HOUSING_R` od středu dílu.

Kolize se kontrolují jen na cestě dolů (body 0–40). Cestou nahoru robot jede po stejné tyči a kontrola tam zbytečně vyřazovala řešení a vedla k nuceným otočkám $q_4$. Omezení z kapitoly 5 (nejvýš 30° za krok, bez přeskoku lokte) platí i cestou nahoru.

### 6.3 Závislost na směru otáčení

Výběr nejbližší konfigurace má nepříjemnou vlastnost: když se kolizní oblast kvůli otáčení desky posouvá k aktuálnímu natočení, robot se odtočí jen o tolik, aby z ní vyklouzl, a jede po hraně kolize. Záleží na směru otáčení stolku, proto robot během vývoje při jednom směru narážel a při druhém ne. To je podstatné pro bonus za oba směry otáčení. Zmírnili jsme to dostatečnou rezervou v `HOUSING_R` (35 mm) a laděním `LEAD_T` zvlášť pro každý směr (tabulka 2).

## 7 Experimenty

Experimenty sloužily k určení parametrů (model doby pohybu, kalibrace, časový předstih) a k ověření, kde řešení funguje. Některé varianty plánovače jsme nejdřív zkoušeli v simulaci: robot nahrazený objektem s kinematikou z knihovny (`RobotBosch(tty_dev=None)`) a dobou pohybu podle modelu, stolek s náhodnou polohou, poloměrem a rychlostí otáčení.

### 7.1 Model doby pohybu

Skript `measure.py` pohnul každým kloubem třikrát o tři různé vzdálenosti ($q_1$, $q_2$: 0,1 / 0,2 / 0,4 rad; $q_3$: 2 / 5 / 10 cm; $q_4$: 0,2 / 0,5 / 1 rad) a měřil dobu od `move_to_q` do konce `wait_for_motion_stop`. Proložení přímkou dalo pro `SPEED` = 0,5 hodnoty v tabulce 4.

| kloub | $a_j$ [s] | $v_j$ |
|---|---|---|
| $q_1$ | 0,214 | 0,396 rad/s |
| $q_2$ | 0,216 | 0,620 rad/s |
| $q_3$ | 0,175 | 0,0263 m/s |
| $q_4$ | 0,135 | 0,518 rad/s |

Tabulka 4: Parametry modelu doby pohybu.

Na šesti náhodných pohybech více kloubů najednou se model od měření lišil nejvýš o 6,1 %. Nejpomalejší je svislý posuv $q_3$: krok 6,7 mm svisle trvá asi 0,43 s, vodorovný krok stejné délky méně.

## 7.2 Přesnost kalibrace

Na 12 kalibračních snímcích jsme porovnali polohu obruče nalezenou v obraze s polohou z kinematiky a nechali dofitovat model obruče. Chyba na snímku vynechaném z fitu (leave-one-out) je v tabulce 5.

| model obruče | střední chyba | max. chyba |
|---|---|---|
| $D = 135$ mm pod 185°, bez dalších oprav | 5,4 mm | 10,1 mm |
| dofitované $D$ | 2,7 mm | 6,1 mm |
| dofitované $D$ + posun nuly kloubu $q_2$ | 0,75 mm | 1,7 mm |

Tabulka 5: Chyba modelu obruče na kalibračních snímcích.

Nejlepší model vyšel $D = 134{,}3$ mm pod $\approx 189°$ a posun $q_2$ o $+3{,}4°$. Chyba asi 1 cm v původním modelu stačí na to, aby obruč minula střed tyče. Finální program proto používá $D$ pod 189° a opravu $\Delta q_2 = 0{,}06$ rad (kapitola 3.2). **[DOPLNIT: jak se oprava projevila na robotu]**

## 7.3 Časový předstih `LEAD_T`

`LEAD_T` se ladil na robotu: když tyč tlačila na přední stranu obruče ve směru otáčení, předstih se zvýšil, a naopak. Chyba časování je nejvíc vidět na částech tyče daleko od osy stolku. Ukázalo se, že nejlepší hodnota závisí na směru otáčení a na tom, na které straně pracovního prostoru stolek leží; výsledné hodnoty jsou v tabulce 2. Závislost na straně je pravděpodobně zbytková chyba geometrie (kalibrace), kterou časový posun částečně vyrovná.

Během ladění jsme pozorovali, že pro stejnou polohu stolku občas fungovalo $+0{,}2$ s i $-0{,}2$ s, ale ne 0. Čistá chyba časování by se takhle chovat nemohla, protože oba posuny posouvají obruč na opačné strany tyče. Příčinou byl rozptyl mezi jednotlivými spuštěními: referenční úhel desky se bral z jediného snímku. Po přechodu na úhel z proložené přímky (kapitola 4.4) a prodloužení focení na 22 snímků byly výsledky stabilnější. **[DOPLNIT: případně tabulka zkoušených hodnot `LEAD_T`]**

## 7.4 Kolize s bludištěm

Na robotu se ukázalo, že do tyče nenarážejí ramena, ale díl u hřídele (kapitola 6). Další zjištění:

- **Otočky o 360°:** bez limitu pootočení $q_4$ si plán občas vybral stejnou polohu s $q_4$ o otočku jinak a robot držákem smetl tyč. Limit 30° na krok to odstranil.
- **Závislost na směru otáčení:** při jednom směru otáčení stolku robot narážel, při druhém ne (vysvětlení v kapitole 6.3).
- **Selhání plánu:** výběr krok po kroku vidí jen jeden krok dopředu. V simulaci na 40 náhodných scénářích plán selhal v 5 případech, přestože bezkolizní řešení existovalo. Proto program při selhání plán jednou zopakuje s novými snímky.

## 7.5 Kde řešení funguje

Zadání doporučuje zobrazit, ve kterých částech pracovního prostoru řešení funguje. Stolek jsme postupně umístili do šesti poloh pro každý směr otáčení a v každé provedli dva pokusy (tabulka 6, obrázek 4). Poloha stolku je střed otáčení vypsaný programem.

| poloha $(x, y)$ [mm] | $\omega$ [°/s] | pokusů | úspěšných |
|---|---|---|---|
| *otáčení* $\omega < 0$ | | | |
| (365, 86) | −12,8 | 2 | 2 |
| (265, 110) | −12,7 | 2 | 2 |
| (390, −21) | −12,9 | 2 | 2 |
| (233, 18) | −12,9 | 2 | 2 |
| (245, −146) | −12,8 | 2 | 2 |
| (388, −162) | −12,7 | 2 | 2 |
| *otáčení* $\omega > 0$ | | | |
| (365, 84) | +13,0 | 2 | 2 |
| (267, 115) | +13,0 | 2 | 2 |
| (411, −18) | +12,6 | 2 | 2 |
| (397, −122) | +12,7 | 2 | 2 |
| (255, −173) | +12,6 | 2 | 2 |
| (232, 17) | +12,6 | 2 | 2 |

Tabulka 6: Úspěšnost řešení v pracovní ploše pro oba směry otáčení.

Řešení uspělo ve všech 24 pokusech, tedy ve všech testovaných polohách v obou směrech otáčení. V poloze nejdál od základny, (411, −18) mm, robot při jednom pokusu zavadil o ochranné sklo pracoviště. Pokus přesto skončil úspěšně, ukazuje to ale, že tato poloha je na hraně bezpečně použitelného prostoru.

# 8 Implementace a spuštění

Řešení je napsané v Pythonu nad knihovnami `ctu_bosch_sr450`, OpenCV a NumPy. Hlavní program se spouští příkazem `python main_finalized.py` z adresáře projektu, kde musí být `duration_model.json` a soubor s homografií.

| soubor | obsah |
|---|---|
| `main_finalized.py` | hlavní program: snímání, plán, kolize, provedení |
| `image_processing.py` | detekce značek, odhad otáčení, předpověď polohy tyče, body tyče |
| `calibrate.py` | kalibrační polohy, výpočet homografie, natočení $q_4$ do středu rozsahu |
| `fit_calibration.py` | fit modelu obruče a posunu $q_2$ z kalibračních snímků |
| `measure.py` | měření modelu doby pohybu |
| `duration_model.json` | změřené parametry $a_j$, $v_j$ |

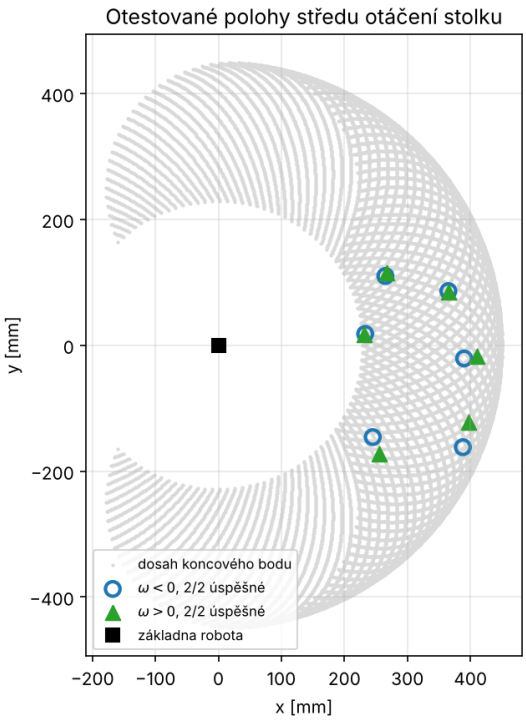
Tabulka 7: Struktura kódu.

Po inicializaci robot natočí $q_4$ na 360° (`fix_negative_uhel`), aby měl na obě strany rezervu na otáčení obruče. Celá jízda je v bloku `try/finally`, takže se robot i při chybě vrátí domů a spojení se uzavře. Když plán nenajde řešení ani na druhý pokus, robot se k bludišti nerozjede.

# 9 Diskuze

Řešení stojí na jednoduchém principu: předpovědět, kde bude tyč, a dojet tam ve správný čas. Přesnost je dána hlavně třemi zdroji chyb:

- **Kalibrace:** model obruče a homografie. Oprava kloubu $q_2$ zmenšila chybu modelu obruče z jednotek milimetrů pod 2 mm (tabulka 5).




Obrázek 4: Otestované polohy středu otáčení stolku na pozadí dosahu koncového bodu robota. Kroužky: $\omega < 0$, trojúhelníky: $\omega > 0$.

- **Odhad otáčení:** chyba úhlové rychlosti roste lineárně s délkou jízdy, chyba fáze se projeví nejvíc daleko od osy stolku.
- **Časování:** stálé zpoždění kamery a robota je vyrovnané ručně laděným `LEAD_T`. Hodnota se nedala odvodit z měření kamerou, protože ta má stejné zpoždění.

Další omezení:

- **Plán bez výhledu dopředu:** výběr konfigurace krok po kroku může skončit v bodě, odkud nevede bezkolizní pokračování do 30°. Dynamické programování přes celou dráhu by to odstranilo (v simulaci 0 selhání ze 40), ale výpočet plánu by trval asi 4 s místo 0,6 s.
- **Zjednodušený model kolize:** hlídá se jen jeden díl u hřídele a jen v cílové poloze kroku, ne během přejezdu mezi kroky. Ochranné sklo pracoviště se nehlídá vůbec.
- **Pohyb se zastavováním:** robot se po každém kroku zastaví. Deska se mezitím otáčí dál, takže mezi kroky obruč nepřesně sleduje tyč.
- **Ruční `LEAD_T` podle strany a směru:** funguje v otestovaných polohách, ale jde o empirickou korekci, ne o odstranění příčiny.

Možná vylepšení: homografie spočítaná se stejným modelem obruče jako plánování, plánování dynamickým programováním, hlídání vzdálenosti od skla, pohyby ve frontě řízení bez zastavování a průběžná oprava fáze stolku z dalších snímků během jízdy.

## 10 Závěr

Robot Bosch SR450 provléká obruč po tyči bludiště na otáčejícím se stolku tak, že z kamery odhadne otáčení desky, předem spočítá časovaný plán konfigurací a ten projede. Plán hlídá limity kloubů, plynulost natočení obruče a kolizi dílu u hřídele s tyčí. Experimenty určily model

doby pohybu, opravu kloubu $q_2$ a časový předstih pro oba směry otáčení. Řešení uspělo ve všech 24 pokusech ve 12 polohách stolku v obou směrech otáčení; v poloze nejdál od základny se robot jednou dotkl ochranného skla. Nedosažitelnou polohu bludiště i chybějící značky program nahlásí před zahájením pohybu.

## 11 Podíl členů týmu na technickém řešení

**[DOPLNIT: celá kapitola]**. Tým má tři členy. Zadání vyžaduje, aby každý člen měl technický přínos a uměl ho při předvádění vysvětlit; rozdělení „implementace vs. psaní zprávy“ nestačí. Návrh rozdělení podle částí řešení je v tabulce 8.

| člen týmu | technická část | konkrétně |
|---|---|---|
| **[DOPLNIT: ]** | kalibrace kamery a obruče | **[DOPLNIT: např. `calibrate.py`, výběr kalibračních poloh, homografie, oprava $q_2$]** |
| **[DOPLNIT: ]** | zpracování obrazu a odhad otáčení | **[DOPLNIT: např. `image_processing.py`, ArUco, fit kružnice a $\omega$]** |
| **[DOPLNIT: ]** | plánování, IK a kolize | **[DOPLNIT: např. `main_finalized.py`, model doby pohybu, kontrola kolize]** |
| **[DOPLNIT: ]** | experimenty a ladění | **[DOPLNIT: např. `measure.py`, ladění `LEAD_T`, mapa pracovní plochy]** |

Tabulka 8: Podíl členů týmu.

## 12 Využití AI při implementaci a psaní zprávy

**[DOPLNIT: ověřit a upravit podle skutečnosti; případně doplnit další nástroje, které tým používal]**

- **Analýza kódu a hledání chyb (Claude):** AI procházela naše skripty a upozorňovala na slabá místa, např. nekonzistentní časování mezi plánem a provedením, chybný směr držáku obruče v kontrole kolize nebo otočky $q_4$ o 360°. To bylo nejpřínosnější použití.
- **Diagnostika z dat:** AI z kalibračních snímků spočítala chybu modelu obruče a navrhla opravu kloubu $q_2$ (tabulka 5). V simulaci ověřovala, kdy plán selže a proč.
- **Návrh kódu:** zjednodušení hlavního programu, kontrola kolize kolidujícího dílu a výběr kalibračních poloh. Geometrické rozměry (poloha dílu, $D$, `Z_PODLOZKA`) a finální hodnoty parametrů jsme měřili a ladili na robotu sami.
- **Psaní zprávy:** první verzi textu napsala AI podle kódu a průběhu vývoje; tým ji zkontroloval a doplnil měření a podíl členů.
- **Zkušenost:** AI byla silná v rychlé analýze a simulaci, ale bez znalosti skutečného robota někdy předpokládala špatnou geometrii (např. že do tyče naráží osa hřídele, ne díl vedle ní). Návrhy proto bylo nutné ověřovat na robotu.